

Java OOP Study Notes: Encapsulation

1. What is Encapsulation?

Encapsulation is the bundling of data (variables) and methods that operate on that data into a single unit (class), while restricting direct access to some components.

Think of it like a **capsule** — everything is wrapped together, and you can only interact with it through controlled openings.

Real-World Analogy

Example	Hidden (Internal)	Exposed (Interface)
ATM Machine	Cash storage, security systems, banking logic	Screen, keypad, card slot
Car	Engine internals, fuel injection, transmission	Steering wheel, pedals, gear shift
TV Remote	Circuit board, infrared signals	Buttons

You don't need to know HOW the ATM counts money internally — you just use the interface.

2. Data Hiding

Data hiding is the principle of hiding internal object details from the outside world. The internal state can only be changed through controlled methods.

Without Data Hiding (Bad Practice)

```
class BankAccount {  
    public double balance; // Directly accessible!  
}  
  
public class Main {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount();  
        acc.balance = 10000;  
  
        // Problems:  
        acc.balance = -5000; // Negative balance? Allowed!  
        acc.balance = 999999999; // Unlimited money? Allowed!  
        // No validation, no logging, no security  
    }  
}
```

With Data Hiding (Good Practice)

```
class BankAccount {
    private double balance; // Hidden from outside

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
        } else {
            System.out.println("Invalid withdrawal");
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();

        // acc.balance = -5000; // ERROR! Cannot access private field

        acc.deposit(10000); // Works - validated
        acc.withdraw(3000); // Works - validated
        acc.withdraw(50000); // Rejected - insufficient funds

        System.out.println("Balance: $" + acc.getBalance());
    }
}
```

```
}  
  
}
```

Output:

```
Deposited: $10000  
Withdrawn: $3000  
Invalid withdrawal  
Balance: $7000.0
```

3. Access Modifiers

Access modifiers control **who can access** a class, variable, method, or constructor.

Overview Table

Modifier	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World
<code>public</code>	✓	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✓	✗
<i>default</i> (no modifier)	✓	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗	✗

Visual Representation



4. `private` Access Modifier

Most restrictive — accessible **only within the same class**.

Use Case

- Internal implementation details
- Sensitive data (passwords, balances)
- Helper methods not meant for external use

Example

```
class User {
    private String password;          // Only this class can access
    private int loginAttempts = 0;

    private boolean validatePassword(String input) { // Private helper
        return input != null && input.length() >= 8;
    }

    public void setPassword(String password) {
        if (validatePassword(password)) {
            this.password = password;
            System.out.println("Password set successfully");
        } else {
            System.out.println("Password must be at least 8 characters");
        }
    }

    public boolean login(String inputPassword) {
        loginAttempts++;
        if (loginAttempts > 3) {
            System.out.println("Account locked!");
            return false;
        }
        if (inputPassword.equals(password)) {
            loginAttempts = 0;
            return true;
        }
        return false;
    }
}

public class Main {
    public static void main(String[] args) {

        User user = new User();

        // user.password = "1234";          // ERROR! private
        // user.validatePassword("test");    // ERROR! private
    }
}
```

```
        user.setPassword("mySecurePass123"); // OK - public method
    }
}
```

5. `default` (Package-Private) Access Modifier

No keyword — accessible within the **same package only**.

Use Case

- Internal package utilities
- Classes that work together within a module
- Implementation classes not meant for public API

Example

File: `com/company/utils/Helper.java`

```
package com.company.utils;

class Helper { // default access - no modifier
    void assist() {
        System.out.println("Helping...");
    }
}
```

File: `com/company/utils/Service.java`

```
package com.company.utils;

public class Service {
    public void doWork() {
        Helper h = new Helper(); // OK - same package
        h.assist();               // OK - same package
    }
}
```

File: `com/company/app/Main.java`

```
package com.company.app;

import com.company.utils.*;

public class Main {
    public static void main(String[] args) {
        // Helper h = new Helper(); // ERROR! Helper is not public

        Service s = new Service(); // OK - Service is public
        s.doWork();
    }
}
```

6. `protected` Access Modifier

Accessible within **same package** AND by **subclasses in any package**.

Use Case

- Methods/fields that subclasses need to override or access
- Template method patterns
- Framework classes meant to be extended

Example

File: `animals/Animal.java`

```
package animals;

public class Animal {
    protected String name;

    protected void breathe() {
        System.out.println(name + " is breathing");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}
```

File: `pets/Dog.java` (different package)

```
package pets;

import animals.Animal;

public class Dog extends Animal {

    public Dog(String name) {
        this.name = name; // OK - protected field accessible in subclass
    }

    public void bark() {
        breathe(); // OK - protected method accessible in subclass
        System.out.println(name + " says: Woof!");
    }
}
```

File: `app/Main.java`

```
package app;

import pets.Dog;
import animals.Animal;

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");

        dog.bark();          // OK - public method
        dog.sleep();         // OK - public method (inherited)

        // dog.name = "Max";   // ERROR! protected - not in subclass or same package
        // dog.breathe();      // ERROR! protected

        Animal animal = new Animal();
        // animal.name = "Cat"; // ERROR! protected
    }
}
```

Output:

```
Buddy is breathing
Buddy says: Woof!
```

7. `public` Access Modifier

Least restrictive — accessible from **anywhere**.

Use Case

- Public API methods
- Entry points (main method)
- Utility methods meant for everyone

Example

```
public class Calculator {  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static int subtract(int a, int b) {  
        return a - b;  
    }  
}  
  
// Accessible from ANY class, ANY package  
int result = Calculator.add(5, 3);
```

8. Access Modifiers Comparison

Complete Example with All Modifiers

```
package com.example;

public class AccessDemo {
    public String publicVar = "I'm public";
    protected String protectedVar = "I'm protected";
    String defaultVar = "I'm default";           // package-private
    private String privateVar = "I'm private";

    public void publicMethod() {
        System.out.println("Public method - accessible everywhere");
        accessAll(); // Can access everything from within class
    }

    protected void protectedMethod() {
        System.out.println("Protected method - package + subclasses");
    }

    void defaultMethod() {
        System.out.println("Default method - same package only");
    }

    private void privateMethod() {
        System.out.println("Private method - this class only");
    }

    // This method can access ALL members (it's in the same class)
    private void accessAll() {
        System.out.println(publicVar);
        System.out.println(protectedVar);
        System.out.println(defaultVar);
        System.out.println(privateVar);
        privateMethod();
    }
}
```

Access from Different Locations

Location	public	protected	default	private
Same class	✓	✓	✓	✓
Same package, different class	✓	✓	✓	✗
Different package, subclass	✓	✓	✗	✗
Different package, non-subclass	✓	✗	✗	✗

9. Getter and Setter Methods

Getters and **setters** are public methods that provide controlled access to private fields.

Naming Convention

Field	Getter	Setter
<code>private int age</code>	<code>public int getAge()</code>	<code>public void setAge(int age)</code>
<code>private String name</code>	<code>public String getName()</code>	<code>public void setName(String name)</code>
<code>private boolean active</code>	<code>public boolean isActive()</code>	<code>public void setActive(boolean active)</code>

Note: Boolean getters use `is` prefix instead of `get`.

Basic Example

```
class Student {  
    private String name;  
    private int age;  
    private boolean enrolled;  
  
    // Getter for name  
    public String getName() {  
        return name;  
    }  
  
    // Setter for name  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    // Getter for age  
    public int getAge() {  
        return age;  
    }  
  
    // Setter for age  
    public void setAge(int age) {  
        this.age = age;  
    }  
  
    // Getter for boolean (uses 'is' prefix)  
    public boolean isEnrolled() {  
        return enrolled;  
    }  
  
    // Setter for boolean  
    public void setEnrolled(boolean enrolled) {  
        this.enrolled = enrolled;  
    }  
}
```

10. Getters & Setters with Validation

The real power of getters/setters is adding **validation** and **logic**.

```
class Employee {
    private String name;
    private int age;
    private double salary;
    private String email;

    // =====
    // NAME - with trimming and validation
    // =====
    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be empty");
        }
        this.name = name.trim();
    }

    // =====
    // AGE - with range validation
    // =====
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age < 18 || age > 65) {
            throw new IllegalArgumentException("Age must be between 18 and 65");
        }
        this.age = age;
    }

    // =====
    // SALARY - with minimum check
    // =====
    public double getSalary() {
        return salary;
    }
}
```

```
public void setSalary(double salary) {
    if (salary < 0) {
        throw new IllegalArgumentException("Salary cannot be negative");
    }
    if (salary < 15000) {
        System.out.println("Warning: Below minimum wage");
    }
    this.salary = salary;
}

// =====
// EMAIL - with format validation
// =====
public String getEmail() {
    return email;
}

public void setEmail(String email) {
    if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\. [a-z]{2,}$")) {
        throw new IllegalArgumentException("Invalid email format");
    }
    this.email = email.toLowerCase();
}
}
```

Usage

```
public class Main {  
    public static void main(String[] args) {  
        Employee emp = new Employee();  
  
        emp.setName(" Aslam Howlader "); // Trimmed automatically  
        emp.setAge(25);  
        emp.setSalary(50000);  
        emp.setEmail("ASLAM@Company.COM"); // Lowercased automatically  
  
        System.out.println("Name: " + emp.getName()); // "Aslam Howlader"  
        System.out.println("Email: " + emp.getEmail()); // "aslam@company.com"  
  
        // Test validation  
        try {  
            emp.setAge(10); // Invalid  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

Output:

```
Name: Aslam Howlader  
Email: aslam@company.com  
Error: Age must be between 18 and 65
```

11. Read-Only and Write-Only Properties

Read-Only (Only Getter)

```
class Order {  
    private final String orderId;  
    private final long createdAt;  
  
    public Order() {  
        this.orderId = "ORD-" + System.currentTimeMillis();  
        this.createdAt = System.currentTimeMillis();  
    }  
  
    // Only getter - no setter (immutable)  
    public String getOrderId() {  
        return orderId;  
    }  
  
    public long getCreatedAt() {  
        return createdAt;  
    }  
}
```

Write-Only (Only Setter) — Rare

```
class AuthService {
    private String apiKey;

    // Only setter - never expose the key
    public void setApiKey(String apiKey) {
        this.apiKey = apiKey;
    }

    // No getter for apiKey!

    public boolean authenticate() {
        // Uses apiKey internally
        return apiKey != null && apiKey.startsWith("sk_");
    }
}
```

12. Computed/Derived Properties

Getters can return **calculated values** that don't have backing fields.

```
class Rectangle {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    // Regular getters
    public double getWidth() { return width; }
    public double getHeight() { return height; }

    // Computed properties (no backing field)
    public double getArea() {
        return width * height;
    }

    public double getPerimeter() {
        return 2 * (width + height);
    }

    public boolean isSquare() {
        return width == height;
    }
}

// Usage
Rectangle rect = new Rectangle(5, 10);
System.out.println("Area: " + rect.getArea());           // 50.0
System.out.println("Perimeter: " + rect.getPerimeter()); // 30.0
System.out.println("Is Square: " + rect.isSquare());     // false
```

13. Complete Encapsulation Example

```
class Product {  
    // Private fields - hidden from outside  
    private String sku;  
    private String name;  
    private double price;  
    private int quantity;  
    private boolean active;  
  
    // Constructor with validation  
    public Product(String sku, String name, double price) {  
        setSku(sku);  
        setName(name);  
        setPrice(price);  
        this.quantity = 0;  
        this.active = true;  
    }  
  
    // =====  
    // GETTERS & SETTERS  
    // =====  
  
    public String getSku() {  
        return sku;  
    }  
  
    public void setSku(String sku) {  
        if (sku == null || !sku.matches("^[A-Z]{3}-\\d{4}$")) {  
            throw new IllegalArgumentException("SKU must be format: ABC-1234");  
        }  
        this.sku = sku;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        if (name == null || name.trim().length() < 2) {
```

```
        throw new IllegalArgumentException("Name must be at least 2 characters");
    }
    this.name = name.trim();
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    if (price < 0) {
        throw new IllegalArgumentException("Price cannot be negative");
    }
    this.price = price;
}

public int getQuantity() {
    return quantity;
}

// No direct setQuantity - use business methods instead

public boolean isActive() {
    return active;
}

public void setActive(boolean active) {
    this.active = active;
}

// =====
// COMPUTED PROPERTIES
// =====

public double getInventoryValue() {
    return price * quantity;
}

public boolean isInStock() {
    return quantity > 0;
}
```

```

}

public String getStatus() {
    if (!active) return "DISCONTINUED";
    if (quantity == 0) return "OUT OF STOCK";
    if (quantity < 10) return "LOW STOCK";
    return "IN STOCK";
}

// =====
// BUSINESS METHODS (controlled modification)
// =====

public void addStock(int amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    this.quantity += amount;
    System.out.println("Added " + amount + " units. New quantity: " + quantity);
}

public void removeStock(int amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    if (amount > quantity) {
        throw new IllegalArgumentException("Insufficient stock");
    }
    this.quantity -= amount;
    System.out.println("Removed " + amount + " units. New quantity: " + quantity);
}

public void applyDiscount(double percentage) {
    if (percentage < 0 || percentage > 50) {
        throw new IllegalArgumentException("Discount must be 0-50%");
    }
    double discount = price * (percentage / 100);
    price -= discount;
    System.out.println("Applied " + percentage + "% discount. New price: $" + price);
}

```

```
// =====  
// DISPLAY  
// =====  
  
public void display() {  
    System.out.println("┌───────────────────┐");  
    System.out.println("│ SKU: " + sku);  
    System.out.println("│ Name: " + name);  
    System.out.println("│ Price: $" + String.format("%.2f", price));  
    System.out.println("│ Quantity: " + quantity);  
    System.out.println("│ Status: " + getStatus());  
    System.out.println("│ Inventory Value: $" + String.format("%.2f", getInventoryValue()));  
    System.out.println("└───────────────────┘");  
}  
}
```

Testing

```
public class Main {  
    public static void main(String[] args) {  
        Product laptop = new Product("LAP-1001", "Gaming Laptop", 1299.99);  
  
        laptop.addStock(25);  
        laptop.display();  
  
        laptop.removeStock(20);  
        laptop.applyDiscount(10);  
        laptop.display();  
  
        // Try invalid operations  
        try {  
            laptop.removeStock(100);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

Output:

Added 25 units. New quantity: 25

```
| SKU: LAP-1001
| Name: Gaming Laptop
| Price: $1299.99
| Quantity: 25
| Status: IN STOCK
| Inventory Value: $32499.75
```

Removed 20 units. New quantity: 5

Applied 10.0% discount. New price: \$1169.991

```
| SKU: LAP-1001
| Name: Gaming Laptop
| Price: $1169.99
| Quantity: 5
| Status: LOW STOCK
| Inventory Value: $5849.96
```

Error: Insufficient stock

14. Summary: Encapsulation Best Practices

Practice	Description
Make fields <code>private</code>	Always start with private, loosen if needed
Use getters/setters	Provide controlled access
Validate in setters	Enforce business rules
Use business methods	<code>addStock()</code> instead of <code>setQuantity()</code>
Computed properties	Calculate on-demand, don't store redundant data
Immutable when possible	Read-only fields for things that shouldn't change

Why Encapsulation Matters

Without Encapsulation	With Encapsulation
<u>object.field = anything</u>	<u>object.setField(value)</u>
▼	▼
No validation	✓ Validation
No logging	✓ Logging
No security	✓ Security
Tight coupling	✓ Loose coupling
Hard to change	✓ Easy to modify

15. Key Points to Remember

1. **Encapsulation** = Data + Methods bundled together with restricted access
 2. **Data Hiding** = Making fields private to protect internal state
 3. **Access Modifiers** control visibility (private → default → protected → public)
 4. **private** = Same class only
 5. **default** = Same package only
 6. **protected** = Same package + subclasses anywhere
 7. **public** = Accessible everywhere
 8. **Getters** retrieve private field values
 9. **Setters** modify private fields with validation
 10. **Boolean getters** use `is` prefix: `isActive()` , `isValid()`
 11. **Computed properties** calculate values without storing them
 12. Prefer **business methods** (`withdraw()`) over direct setters (`setBalance()`)
- ```
' ; triggerDownload(docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html; charset=utf-8'); }
function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); //
Force light mode for PDF (dark backgrounds print poorly) const wasDark =
```

```

document.documentElement.classList.contains('dark'); if (wasDark)
document.documentElement.classList.remove('dark'); const element =
document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename:
docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale:
1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' },
pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if
(wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid
rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme:
dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre
code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !==
'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try {
mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i <
mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const
code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const
container = document.createElement('div'); container.className = 'mermaid-container';
container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render
failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function
updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark;
document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */
window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const
isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);
updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme:
isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} });

```